AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH
Faculty of Science and Technology
Department of Computer Science
CSC 3215: Web Technologies

Semester: Fall 2025-26 **Group No:-** **Section:N**

Group Members:

SL.	Student ID	Name	Role	Signature
1	SADIA AHMED	23-55053-3	Member	
2	ORPYTA KARMOKAR AUPY	23-55051-3	Member	
3	TASMIA AMIN	23-53122-3	Member	

Project Title

Doraemon's Treat Hunt: A 2D Side-Scrolling Game in OpenGL

Short Description

Doraemon's Treat Hunt is a 2D side-scrolling game written in C++ with OpenGL and GLUT. In each of its three levels the player guides Doraemon through a scrolling town to collect five dorayaki while avoiding level-specific hazards. Every object, from the character to the houses, is drawn procedurally from OpenGL primitives and parametric curves. This report describes the gameplay and each level in detail: its scene, lighting, hazards and the functions that build it. It also gathers every mouse, keyboard and GLUT function the game uses in one place.

1. Introduction

The project brings together the main topics of a computer graphics course in one interactive program. A side-scrolling game was chosen because it combines drawing, 2D transformations, animation, lighting, blending and real-time user input in a single loop.

The game follows the familiar cartoon character Doraemon, whose favourite food is dorayaki. Three levels show the same town at different times of day: a sunny day, a sunset and a night. Each time of day has its own lighting and its own kind of hazard.

The whole game is written in one C++ file and uses OpenGL for drawing and GLUT for the window, keyboard and mouse.

2. Game Description

A level is won by collecting 5 dorayaki and lost when all 3 lives are gone. Each level is a street of five scenes, 23 units wide each, alternating a house scene and a wall scene. When Doraemon walks past x = −6 the street scrolls left instead, so he stays near the left of the screen.

2.1 Elements common to every level

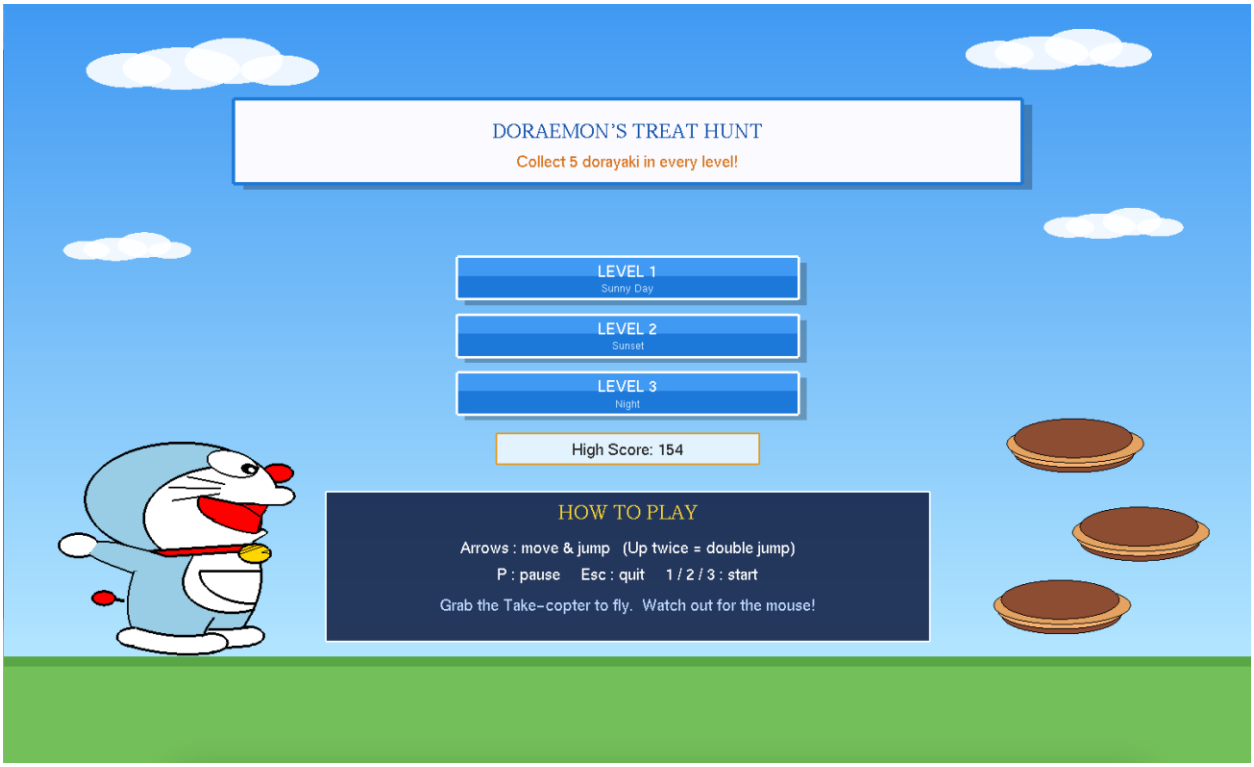
Element	Behaviour	Functions
Doraemon (player)	Walks, jumps with gravity, double-jumps; blinks for 45 frames after a hit	`Doraemon()`, `drawPlayer()`, `update()`, `tryJump()`
Street scenery	House scene and wall scene repeated along the street	`houseScene()`, `wallScene()`, `drawHome()`, `drawwall()`, `drawBox()`, `Tree1()`, `drawFlower()`, `electricPole()`
Dorayaki (5)	Collected on touch: +10 points, sparkle and sound	`placeCollectibles()`, `Doracake()`, `drawEllipse()`, `checkCollectibles()`
Hearts (3)	Restore one lost life when touched	`placeCollectibles()`, `Heart()`, `checkCollectibles()`
Take-copter	Power-up: 5 seconds of flight, Up to rise, release to glide	`placeGadget()`, `drawGadgetItem()`, `drawTakeCopter()`, `checkGadget()`
Mouse enemy	Patrols the ground; Doraemon is afraid of it and runs the other way	`placeMouseEnemy()`, `updateMouseEnemy()`, `drawMouseEnemy()`, `drawMouse()`
Damage	Removes a life, starts invincibility, red flash and particles	`takeHit()`

Element	Behaviour	Functions
Effects	Sparkles, plus short sounds on collect, hit, level complete, game over, Take-copter pickup and mouse scare (macOS system sounds; .wav files on Windows)	`spawnParticles()`, `updateParticles()`, `drawParticles()`, `playSound()`
HUD	Cakes, lives, level, time, score, high score, progress bar	`drawHUD()`, `levelProgress()`, `drawOverlays()`
Level setup	Resets player, world, items and hazards together	`startLevel()`, `resetLevel()`, `restartLevel()`

2.2 Scoring

Each dorayaki gives 10 points. Finishing a level in under 60 seconds adds 2 points for every second left. The score carries over to the next level, and the best score is saved in `highscore.txt` by `saveHighScore()` and read back by `loadHighScore()`.

Figure 1. Homepage screenshot



3. Level 1: Sunny Day

Level 1 is the introductory level: a bright daytime street whose only hazards are three stationary stones on the road.

Figure 2. *Level 1 gameplay screenshot*



3.1 Scene and lighting

The sky is a gradient from deep blue at the top to light blue near the horizon. A sunlight-style light keeps all colours bright and even.

3.2 Hazards

Three stones are placed at random positions along the street, at least 5 units apart, resting on the ground (y = −7.5). Each stone is an orange polygon shape with Bézier-curve outlines, scaled down to about 1 unit. Touching a stone costs one life; the player avoids it by jumping.

3.3 Functions used

Function	Role in Level 1
`Level1()`	Draws the whole level each frame, in back-to-front order
`drawLighting1()`	Enables directional daylight `GL_LIGHT0` and switches off `GL_LIGHT1`
`drawSky1()`	Draws the blue gradient sky
`houseScene()`, `wallScene()`	Draw the five street scenes, offset by the scroll value `worldX`

Function	Role in Level 1
`placeStones()`	Places the 3 stones at random, at least 5 units apart
`drawStones()`, `drawStone()`	Draw each stone at its scrolled position
`checkStones()`	Detects a stone collision and calls `takeHit()`
`drawPlayer()`, `Doracake()`, `Heart()`	Draw Doraemon, the dorayaki and the hearts
`drawGadgetItem()`, `drawMouseEnemy()`	Draw the Take-copter and the mouse
`drawOverlays()`	Draws particles, hit flash, HUD, pause and end screens on top

4. Level 2: Sunset

Level 2 adds moving hazards and timing: four rotating sign boards drift around the street and three manhole fires flare up on a fixed cycle.

Figure 3. *Level 2 gameplay screenshot*



4.1 Scene and lighting

The sky fades from blue-grey at the top to sunset orange at the horizon. A warm light placed above the scene dims and warms everything, including Doraemon, to give an evening mood.

4.2 Hazards

- **Rotating sign boards (4).** Circular boards of radius 1.5 that spin 3° per frame. Each board is given a random movement: about one in three slides left and right across its scene, and vertically it either bounces between y = −7 and −4, falls from y = −1 to the ground and rises again, or stays still.
- **Manhole fires (3).** A dark manhole on the road that shoots a flame for 60 frames (about 1 second), then rests for 60 frames. The flame height flickers with time (1.2 ± 0.2 units). Stepping over a burning manhole costs a life; the player must time the crossing or jump.

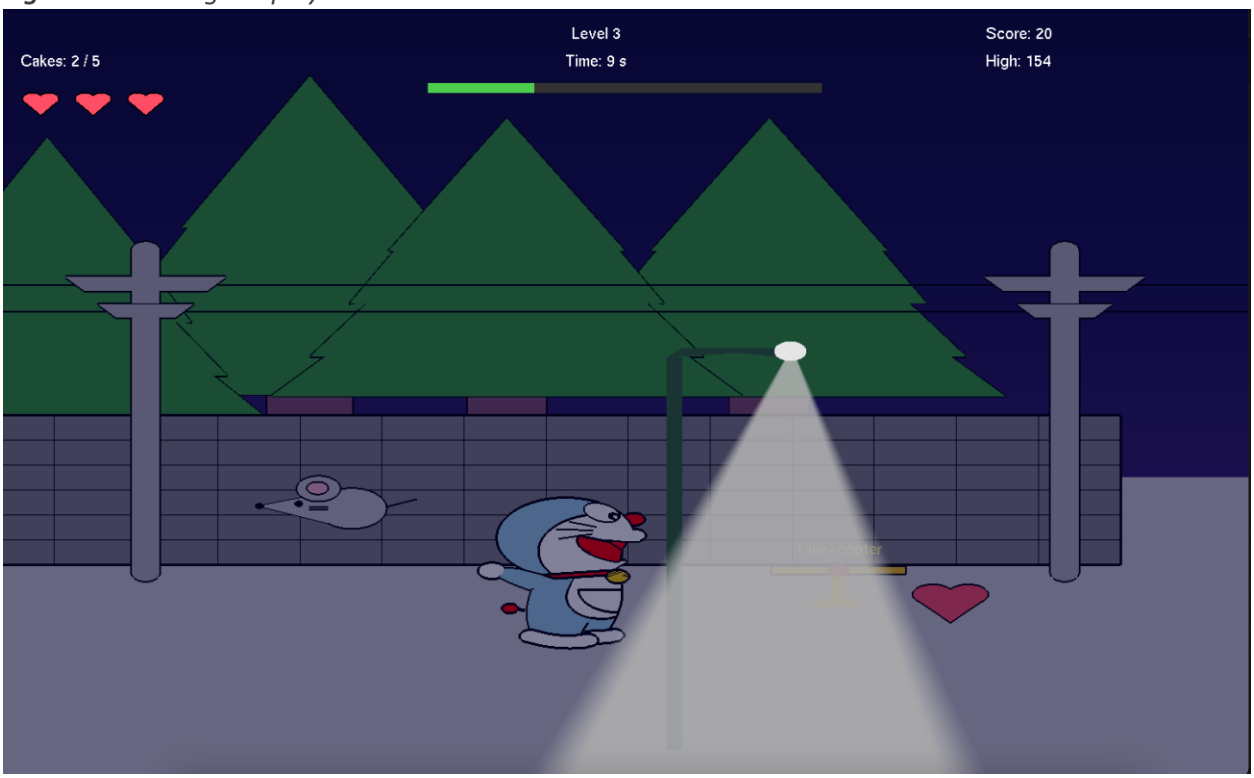
4.3 Functions used

Function	Role in Level 2
`Level2()`	Draws the whole level each frame
`drawLighting2()`	Enables warm positional light `GL_LIGHT1` and switches off `GL_LIGHT0`
`drawSky2()`	Draws the sunset gradient sky
`houseScene()`, `wallScene()`	Draw the scrolling street scenes
`placeObstacles()`	Places the 4 boards at least 6 units apart and picks each one's movement
`updateObstacles()`	Moves each board and advances its rotation angle
`drawRotatingSignBoard()`, `signBoard()`	Rotate and draw a board about its centre
`getObstacleHalfSize()`, `checkObstacles()`	Board collision box and hit test
`placeManholeFire()`	Places the 3 manholes at random along the street
`updateManholeFire()`	Runs each fire's 60-frame on/off timer
`drawManholeFire()`	Draws the manhole and, when on, the flickering flame
`checkManholeFire()`	Detects contact with a burning fire and calls `takeHit()`
`drawPlayer()`, `Doracake()`, `Heart()`, `drawGadgetItem()`, `drawMouseEnemy()`, `drawOverlays()`	Shared elements, as in Level 1

5. Level 3: Night

Level 3 is the hardest level: the street is dark, lit only by street lamps, and four obstacles of three different kinds move around it.

Figure 4. *Level 3 gameplay screenshot*



5.1 Scene and lighting

The sky fades from near-black blue to a faint purple at the horizon. Normal lighting is switched off; instead a see-through dark-blue layer is drawn over the whole street to make it look like night. The street lamps are drawn on top of that layer, so they stay bright, and each has a soft cone of light.

5.2 Hazards

Four obstacles are placed at least 6 units apart, each randomly one of three types. They use the same movement patterns as the Level 2 boards: sliding sideways, bouncing, falling from above, or staying still.

- **Ball.** A pink circle of radius 1, drawn as a triangle fan with a darker centre.
- **Mouse obstacle.** The mouse drawing at three times its normal size. Unlike the mouse enemy, touching it costs a life.
- **Spiky fruit.** A black circle surrounded by 20 triangular spikes.

5.3 Functions used

Function	Role in Level 3
`Level3()`	Draws the whole level each frame, with lighting turned off
`drawSky3()`	Draws the night gradient sky
`houseScene()`, `wallScene()`	Draw the scrolling street scenes

Function	Role in Level 3
`drawLampPost()`	Draws each street lamp and its translucent light cone
`placeObstacles()`	Places the 4 obstacles, picks each type and movement
`updateObstacles()`	Moves each obstacle along its path
`drawBall()`	Draws the ball obstacle
`drawMouse()`	Draws the mouse obstacle (scale 3)
`drawSpikyFruit()`, `drawCircle()`	Draw the spiky fruit obstacle
`getObstacleHalfSize()`, `checkObstacles()`	Collision box per type and hit test
`drawPlayer()`, `Doracake()`, `Heart()`, `drawGadgetItem()`, `drawMouseEnemy()`, `drawOverlays()`	Shared elements, as in Level 1

6. Menus and Screens

Three screens frame the levels: the home menu before play, the pause panel during play and the end screen after a level. All of them centre their text by measuring it with `glutBitmapLength`, so the layout stays aligned at any window size.

> ***Figure 4.*** *Home menu screenshot (insert here)*

The **home menu** shows a sky gradient with clouds and grass, a white title banner, three level buttons (Sunny Day, Sunset, Night), the saved high score and a How to Play panel. Doraemon stands on the left and three dorayaki sit on the right. Lighting is turned off here so the menu colours are never tinted by the last level played.

> ***Figure 5.*** *Pause screen screenshot (insert here)*

The **pause panel** dims the frozen game and shows how to resume (P) or return to the menu (Esc).

> ***Figure 6.*** *End screen screenshot (insert here)*

The **end screen** shows “LEVEL COMPLETE!”, “YOU WIN! ALL LEVELS DONE!” or “GAME OVER” with the score, time bonus and high score. It offers Next Level (after Levels 1 and 2 only), Main Menu and Restart buttons.

Function	Role
`homeScreen()`	Draws the complete home menu
`drawHomeButton()`	Draws one level button with label and subtitle

Function	Role
`drawCloud()`, `fillCircle()`	Draw the clouds from overlapping ellipses
`drawRect()`, `drawRectOutline()`	Filled and outlined rectangles for panels and buttons
`drawEndScreen()`, `drawPanelButton()`	Draw the end-screen panel and its buttons
`drawFullScreenQuad()`	Translucent full-screen layer for the pause dim and red hit flash
`drawText()`, `drawTextFont()`, `drawTextCentered()`, `textWidth()`	Bitmap text output and centring

7. User Input and GLUT Functions

The game is controlled with the arrow keys, four ordinary keys and the left mouse button; GLUT delivers each of these to a callback registered in `main()`.

7.1 Keyboard: arrow keys (special keys)

Handled by `specialKeyPress()` (key down) and `specialKeyRelease()` (key up), registered with `glutSpecialFunc` and `glutSpecialUpFunc`. Key repeat is turned off with `glutIgnoreKeyRepeat(1)`, so a held key sends one press and one release.

Key	Action	Helper functions
Right arrow (`GLUT_KEY_RIGHT`)	Move right while held	`updateMovementFromKeys()`
Left arrow (`GLUT_KEY_LEFT`)	Move left while held	`updateMovementFromKeys()`
Up arrow (`GLUT_KEY_UP`)	Jump; press again in the air to double-jump; hold to rise while flying the Take-copter	`tryJump()`

7.2 Keyboard: ordinary keys

Handled by `keyPress()`, registered with `glutKeyboardFunc`. The same key can mean different things depending on the current screen.

Key	Home menu	During play	End screen
1 / 2 / 3	Start Level 1 / 2 / 3	—	—
Enter	Start Level 1	—	Next Level, or Restart if there is none
P	—	Pause or resume	—

Key	Home menu	During play	End screen
R	—	—	Restart the level
Esc	Quit the game	Return to the menu	Return to the menu

7.3 Mouse

Handled by `mouseClick()`, registered with `glutMouseFunc`. Only the left mouse button is used. When the player clicks, the game checks which button was under the pointer and runs its action.

Screen	Clickable buttons	Function called
Home menu	Level 1, Level 2, Level 3	`startLevel()`
End screen	Next Level	`goToNextLevel()`
End screen	Main Menu	`goToMainMenu()`
End screen	Restart	`restartLevel()`

7.4 All GLUT and setup functions

Function	Purpose
`glutInit()`	Initialise GLUT
`glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)`	Double-buffered RGB window
`glutInitWindowSize(1500, 1000)`	Initial window size in pixels
`glutCreateWindow("Doraemon's Treat Hunt")`	Create the window and its title
`init()` with `gluOrtho2D(-9.5, 9.5, -10, 10)`	Set the 2D orthographic projection
`glutDisplayFunc(display)`	Register the drawing callback, which calls the home menu or the current level
`glutTimerFunc(16, update, 0)`	Run `update()` every 16 ms (about 60 frames per second)
`glutSpecialFunc`, `glutSpecialUpFunc`	Register the arrow-key callbacks
`glutKeyboardFunc`	Register the ordinary-key callback
`glutIgnoreKeyRepeat(1)`	Disable automatic key repeat
`glutMouseFunc`	Register the mouse callback

Function	Purpose
`glutMainLoop()`	Start the event loop
`glutPostRedisplay()`	Request a redraw after each update
`glutSwapBuffers()`	Show the finished frame (double buffering)
`glutGet(GLUT_ELAPSED_TIME)`	Time source for flame flicker and the bobbing Take-copter
`glutBitmapCharacter()`, `glutBitmapLength()`	Draw bitmap text and measure its width

8. Conclusion

Doraemon's Treat Hunt shows how one set of OpenGL techniques can produce three clearly different levels. The same street becomes a bright day with directional light, a warm sunset with a positional light, and a dark night built from an alpha-blended overlay and glowing lamp cones. Difficulty rises from Level 1's still stones to Level 2's spinning boards and timed fires, then to Level 3's mixed moving obstacles in low visibility. Shared elements give every level the same core play: dorayaki, hearts, the Take-copter, the mouse enemy and the HUD. Keyboard and mouse input reach the game through GLUT callbacks that only change state, keeping drawing and logic cleanly separated.